

검진팀 실습 — 스케줄 앱에 “질문하기”(자연어 질의) 붙이기

4주차 핵심: **모델은 공유 게이트웨이(:11500)가 도맡습니다.** 추론 누출 차단(think:false)·출력 길이 상한(num_predict)·모델명 정규화를 게이트웨이가 알아서 해 주므로, 여러분은 **프론트 (맥락 + 규칙)** 작성에만 집중하면 됩니다. 이것이 “하네스 실습”입니다.

이 앱은 기존 일정표(만들기·엑셀 내보내기/불러오기)는 **그대로 둡니다.** “물어보면 답해 주는” AI 한 칸만 더합니다.

1. 내 앱 한눈에 — 무엇을 추가하나

- **지금 앱:** 종합검진센터 일정표를 화면에서 만들고, 엑셀로 내보내고/불러오는 도구입니다. 백엔드는 파이썬 표준 라이브러리(`http.server`) 하나로 화면(`index.html`)과 API(`/api/...`)를 **같은 주소(:3000)**에서 함께 서빙합니다. (그래서 별도 프론트 서버·CORS 설정이 필요 없습니다.)
- **추가할 것 — 딱 2가지:**
 1. **화면 하단에 질문 한 칸:** 질문 입력칸 + [질문] 버튼 + 답변 영역 .
 2. **백엔드에 엔드포인트 1개:** `POST /api/ask` . 화면이 들고 있는 **오늘 배정 스냅샷** (엑셀 내보내기에 쓰는 그 JSON)과 질문을 받아, 게이트웨이에 물어보고 답을 돌려줍니다.
- **예시 질문:** “오늘 오후 반휴자 누구야?”, “내시경실 담당 누구야?”, “휴가 몇 명이야?”
- **하네스 4요소로 보면:**
 - 맥락 = 오늘 배정 스냅샷(검사실·시간대·담당·휴가)
 - 규칙 = “배정표에 있는 내용만으로 답하라. 없으면 ‘표에 없습니다.’”(환각 차단)
 - 도구 = 게이트웨이 `POST /api/chat`
 - 검증 = 표에 실제로 있는 사실만 답하는지, 없는 걸 지어내지 않는지

2. 라우팅 한 곳 추가 (복붙 코드)

손델 파일은 백엔드 `server.py` 한 곳입니다. 아래 3개를 순서대로 붙여 넣으면 끝입니다.

2-1. 게이트웨이 호출 헬퍼 (파일 맨 위 `import` 줄들 근처에 붙여넣기)

이 앱은 `requests` / `httpx` 같은 외부 라이브러리가 없습니다. 그래서 **표준 라이브러리 `urllib`**로 게이트웨이를 부릅니다. 아래는 그대로 복사하세요. (게이트웨이가 `think·num_predict`를 처리하므로 우리가 신경 쓸 건 없습니다.)

```
import urllib.request # 파일 상단 import 구역에 함께 두세요 (json은 이미 import되어 있음)
```

```
GATEWAY = "http://121.138.151.6:11500" # 수업용 공유 게이트웨이 (:11434 아님!)
```

```
def ai_ask(system: str, user: str) -> str:
```

```

body = {
    "model": "qwen3.6:35b-a3b",
    "messages": [
        {"role": "system", "content": system},
        {"role": "user", "content": user},
    ],
    "stream": False,
}

req = urllib.request.Request(
    f"{GATEWAY}/api/chat",
    data=json.dumps(body).encode("utf-8"),
    headers={"Content-Type": "application/json"},
)

with urllib.request.urlopen(req, timeout=120) as r:
    return json.loads(r.read())["message"]["content"]

```

★ 4주차의 “URL 한 줄” 정신: 주소가 :11434 (각자 띄운 모델)가 아니라 :11500 (공유 게이트웨이) 인지 꼭 확인하세요. 이 한 곳이 운영급 차이를 만듭니다.

2-2. do_POST 에 분기 한 줄 추가 (446줄 근처)

지금 do_POST 는 이렇게 생겼습니다. else 바로 위에 분기 한 줄을 끼워 넣으세요.

```

def do_POST(self):
    path = self.path.split("?", 1)[0]
    if path == "/api/export-excel":
        self._handle_export()
    elif path == "/api/import-excel":
        self._handle_import()
    elif path == "/api/staff-profiles":
        self._handle_profiles_post()
+   elif path == "/api/ask":
+       self._handle_ask()
    else:
        self._send_json(404, {"error": "Not found"})

```

2-3. 핸들러 추가 (_handle_import 같은 다른 _handle_* 메서드들 옆에 붙여넣기)

본문 읽기 _read_body() (457줄), JSON 응답 _send_json() (461줄)은 이미 있는 헬퍼입니다. 그대로 씁니다. 아래에서 system / user 두 줄이 여러분이 채울 자리(TODO)입니다. 일단 통째로 붙여 넣고, 3번에서 채웁니다.

```

# POST /api/ask body: {"question": "...", "snapshot": {<오늘
배정 JSON>}}

def _handle_ask(self):
    payload = json.loads(self._read_body() or b"{}")
    question = payload.get("question", "")

```

```
snapshot = payload.get("snapshot", {})
```

```
# — TODO(학생): 아래 두 줄을 채우세요 (3번 참고)
system = "" # 규칙: "배정표 안에서만 답하라" 등
user = "" # 맥락(배정표) + 질문을 함께
#
```

```
answer = ai_ask(system, user)
self._send_json(200, {"answer": answer})
```

3. 학생이 채울 프롬프트 영역 (★ 이게 오늘의 진짜 실습)

`system` (규칙)과 `user` (맥락 + 질문)를 어떻게 쓰느냐가 답의 품질을 가릅니다. 아래 힌트를 보고 **TODO** 두 줄을 채우세요.

system — 규칙을 박는 곳 (핵심 5~8줄)

힌트로 채울 항목: - “아래 **배정표 JSON에 있는 내용만으로** 답하라.” - “표에 없으면 지어내지 말고 **‘표에 없습니다’** 라고만 답하라.” (환각 차단 — 필수) - “답은 **짧고 명확하게**, 이름·검사실·시간대 위주로.” - “추측·일반 상식·표 밖 정보는 쓰지 말 것.”

채움 예시(이대로 써도 되고, 더 다듬어도 됩니다):

```
system = (
    "너는 종합검진센터 일정 담당자를 돕는 비서다.\n"
    "아래 배정표 (JSON)에 있는 내용만 근거로 답하라.\n"
    "표에 없는 것은 지어내지 말고 '표에 없습니다'라고만 답하라.\n"
    "답은 짧고 명확하게, 이름과 검사실·시간대 위주로 답하라.\n"
    "표 밖의 일반 상식이나 추측은 쓰지 마라."
)
```

user — 맥락(배정표) + 질문을 넣는 곳

힌트: 배정표 JSON을 한글이 깨지지 않게(`ensure_ascii=False`) 문자열로 만들어 질문과 함께 넣습니다.

```
user = (
    "[배정표]\n"
    + json.dumps(snapshot, ensure_ascii=False, indent=2)
    + f"\n\n[질문] {question}"
)
```

다 채웠다면 핵심 학습 포인트는 이것입니다: **모델 설정(think·길이·모델명)은 하나도 안 건드렸는데** 깔끔한 답이 나옵니다. 그 일을 게이트웨이가 대신 했기 때문입니다.

4. 화면에 질문 한 칸 추가 (프론트)

index.html 의 본문 맨 아래(닫는 `</body>` 직전 등 잘 보이는 곳)에 아래를 붙여 넣습니다. 백엔드와 같은 주소라서 fetch는 상대경로 `'/api/ask'` 면 됩니다.

```
<!-- — AI 질문하기 칸 ———— -->
<div style="margin:16px; padding:12px; border:1px solid #ccc;
border-radius:8px;">
  <strong>일정표에 질문하기</strong>
  <div style="display:flex; gap:8px; margin-top:8px;">
    <input id="askInput" type="text" placeholder="예: 오늘 오후 반휴자
누구야?"
      style="flex:1; padding:6px;">
    <button id="askBtn">질문</button>
  </div>
  <div id="askAnswer" style="margin-top:10px; white-space:pre-
wrap; color:#222;"></div>
</div>

<script>
document.getElementById("askBtn").addEventListener("click", async
() => {
  const question =
document.getElementById("askInput").value.trim();
  const out = document.getElementById("askAnswer");
  if (!question) return;
  out.textContent = "물어보는 중...";

  // ▼ 오늘 배정 스냅샷: 엑셀 내보내기에 쓰는 그 JSON을 그대로 넣으세요.
  //   (이미 화면이 들고 있는 값. 실습에서는 5번의 '더미 스냅샷'으로 시작해도 됩니
다.)
  const snapshot = window.__snapshot || {}; // ← 본인 앱의 스냅샷 변
수로 연결

  try {
    const res = await fetch("/api/ask", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ question, snapshot }),
    });
    const data = await res.json();
    out.textContent = data.answer || data.error || "(응답 없음)";
  } catch (e) {
    out.textContent = "오류: " + e;
  }
});
</script>
```

`snapshot` 은 “이미 화면이 들고 있는 당일 배정 JSON”입니다. 어느 변수인지 모르면, 일단 **5번의 더미 스냅샷**을 `window.__snapshot` 에 넣어 두고 동작부터 확인한 뒤, 나중에 실제 스냅샷 변수로 연결하면 됩니다.

5. 확인 절차

순서대로 복붙하면 됩니다.

1) 게이트웨이가 살아 있는지 먼저 확인

```
curl http://121.138.151.6:11500/health
```

→ 정상 응답이 오면 OK. (응답이 없으면 강사에게 알리고 잠시 대기하세요.)

2) 내 앱(서버) 기동 — `server.py` 가 있는 폴더에서:

```
python server.py
```

→ 서버 실행 중: `http://localhost:3000` 이 보이면 정상. 브라우저에서 `http://localhost:3000` 을 엽니다.

3) 엔드포인트 단독 테스트 (화면 없이 백엔드만 — 아래 더미 스냅샷 사용):

```
curl -X POST http://localhost:3000/api/ask \
-H "Content-Type: application/json" \
-d '{
  "question": "오후 반휴자 누구야?",
  "snapshot": {
    "date": "2026-06-29",
    "검사실": {
      "내시경실": [{"name": "가가가", "time": "전일"}],
      "초음파실": [{"name": "나나나", "time": "오전"}]
    },
    "휴가": [{"name": "다다다"}],
    "반휴": [{"name": "라라라", "time": "오후"}]
  }
}'
```

→ 기대 답: “라라라” (반휴·오후가 라라라 한 명). `{"answer": "..."}` 형태로 돌아옵니다.

4) 환각 차단(없는 것 묻기) 테스트 — 표에 없는 걸 물어보세요:

```
curl -X POST http://localhost:3000/api/ask \
-H "Content-Type: application/json" \
-d '{"question": "MRI실 담당 누구야?", "snapshot": {"검사실": {"내시경실": [{"name": "가가가"}]}}}'
```

→ 기대 답: “**표에 없습니다**”. (MRI실은 스냅샷에 없으므로 지어내면 안 됩니다. 이게 나오면 규칙이 잘 박힌 것입니다.)

5) think(추론) 누출 없는지 확인 ★

- 답변에 `<think>...</think>`, “사용자가 묻는 것은...” 같은 **혼잣말이 섞여 나오지 않으면 정상**입니다.
- 게이트웨이(:11500)가 `think:false` 를 자동 적용하기 때문입니다. 만약 혼잣말이 보이면 → 주소가 `:11434` 로 잘못 박혔을 가능성. 2-1의 `GATEWAY` 값을 다시 확인하세요.

검증 체크리스트 - [] `/api/ask` 가 200 + `answer` 를 반환한다. - [] 표에 있는 사실(담당·휴가·시간대)을 정확히 답한다. - [] 표에 없는 질문에 “표에 없습니다”로 답한다(환각 차단 동작). - [] **더미 데이터만** 썼다(실명·사번 미전송 확인). - [] 기존 일정표·엑셀 내보내기/불러오기가 그대로 동작한다(회귀 없음).

6. 안전수칙 (이 과제 특화 — 꼭 지켜세요)

- ⚠ 이 앱의 `data/staff-profiles.json` 에는 실명과 사번(`empId`) 자리가 있습니다 = 개인정보(PII). 실습·시연은 반드시 **더미·비식별** 스냅샷으로 하세요. 실제 직원 이름·사번을 게이트웨이(외부 공유 모델)로 보내지 마세요.
 - 더미 이름 예: `가가가`, `나나나`, `홍길동`, `직원A`. 사번·전화번호는 비우거나 가짜로.
 - 위 5번의 더미 스냅샷을 출발점으로 쓰면 안전합니다.
- “**표에 없으면 표에 없습니다**” 규칙은 필수입니다(3번 `system`). 환각으로 없는 사람을 배정하면 운영 사고로 이어집니다.
- **공유 GPU 1개 = 동시성 주의**: 게이트웨이 뒤 GPU는 다 같이 한 대를 씁니다. 여러 명이 동시에 [질문]을 누르면 느려집니다. **한 명씩 천천히** 시연하세요.
- 게이트웨이 경유라 추론 토큰 누출은 없습니다. 그래도 **답을 그대로 신뢰하지 말고**, 표(스냅샷)와 대조해 확인하는 습관을 들이세요(검증).
- 실제 배정표로 운영에 쓰려면, 게이트웨이가 **사내망 안에** 있는지 먼저 확인하고 강사·전산팀과 상의하세요. 오늘 실습은 더미까지입니다.